Customer Support Complaint Classification: TF-IDF vs. Transformer Embeddings

A data science project comparing a classical NLP baseline against a modern transformer-embeddings approach, on the same fair comparison, to test whether the modern approach's accuracy gain is worth its extra compute cost.

---

1. Team & Roles

Name	Role
Rayyan Ahemad Mohd	Exploratory Data Analysis
Ebelechukwu Chinedu Umezude	Classical Baseline (TF-IDF + Logistic Regression)
Sai Prasad Bandari	Transformer Approach (Sentence Embeddings + Logistic Regression)

---

2. Project Goal

Customer support teams receive high volumes of complaint tickets that need to be routed to the right category (Payments, Shipping, Refunds, etc.). This project builds and compares two ways of automatically classifying complaints into 16 categories:

1. **Classical approach** — TF-IDF (word-count features) ⟶ Logistic Regression
2. **Transformer approach** — pretrained sentence-transformer embeddings (meaning-based features) ⟶ the **same** Logistic Regression classifier

The goal is **not** simply "which approach scores higher" — it's answering a more useful engineering question: **is the transformer approach's accuracy gain (if any) large enough to justify its extra computational cost**, particularly on the rare and semantically overlapping categories where the two approaches are expected to differ most?

---

3. Dataset

Source: `ecommerce_customer_support_cleaned.csv`, matching `Ataur77/ecommerce-customer-support` on Hugging Face.

- **1,334 complaints**, 3 columns: `Complaint` (text), `Category` (16 labels, the classification target), `Urgency` (4 levels, not used for classification but explored in EDA)
- No missing values, no duplicate rows

- Chosen over larger alternatives (e.g. the CFPB complaints dataset) specifically because the transformer approach's embedding-generation step is the compute bottleneck — a smaller dataset keeps that step manageable while still being large enough for a meaningful comparison

---

4. Hypothesis

> The transformer approach will outperform the classical baseline mainly on the **rare** and **semantically overlapping** categories, but at a **compute cost that may not be justified** for the common, easy-to-separate categories.

---

5. Research Angle (Originality)

This project's framing extends beyond a simple accuracy comparison in three specific ways:

1. **A genuinely fair comparison** — both approaches use the exact same train/test split and the exact same downstream classifier, isolating the feature-representation method as the only variable under test.
2. **Explicit handling of class imbalance** — rather than ignoring the dataset's uneven category sizes, both notebooks measure and report how much `class_weight='balanced'` specifically helps rare categories vs. common ones.
3. **Deeper error analysis** — beyond a single accuracy number, both notebooks include confusion matrices, per-class precision/recall/F1, and manual inspection of misclassified complaints, particularly for rare categories.

---

6. Method Decisions (Fairness Rules)

These rules are applied identically across the EDA, baseline, and transformer notebooks:

- **One shared 80/20 stratified split, `random_state=42`.** The transformer notebook derives this split directly; the baseline notebook loads the exact same row indices from `Transformer/shared_split/` rather than re-deriving it, to guarantee row-for-row identical train/test sets.
- **All 16 `Category` labels kept unmerged** — merging near-duplicate categories (e.g. "Order Issues" and "Order Tracking") would erase the rare-class phenomenon this project studies.
- **Macro F1 as the primary metric**, not plain accuracy — macro F1 averages performance across all 16 categories equally, so failures on rare categories aren't hidden by strong performance on common ones (which plain accuracy would allow, since common categories dominate the total sample count).
- **Compute cost tracked alongside accuracy** — vectorization/embedding time, classifier training time, inference latency, and (for the transformer) parameter count and memory usage.

---

7. Exploratory Data Analysis — Key Findings

(Full analysis in `EDA/eda_customer_complaints.ipynb`)

- **Significant class imbalance confirmed:** "Promotions" is the largest category (137 complaints), "Refunds" the smallest (20 complaints) — a **6.8x imbalance** between the biggest and smallest category. This directly motivates the project's use of macro F1 and class weighting.
- **Category and Urgency are statistically associated**, not independent (chi-square test: $\chi^2 = 154.22$, df = 45, $p < 0.0001$).
- **Complaint length varies meaningfully by category** — some categories skew toward longer, more detailed complaints.
- **Vocabulary overlap between related categories is real and measurable:** "Payments" and "Checkout" complaints share **29.8%** of their vocabulary (words like "card," "verification," "blocked," "mismatch"). This is precisely the kind of semantically overlapping category pair where the project's hypothesis expects the transformer's contextual understanding to help more than TF-IDF's exact word-matching.

---

8. Classical Baseline — TF-IDF + Logistic Regression

(Full notebook: `Baseline/tfidf_baseline_classification.ipynb`)

Method: Each complaint is converted to a vector of TF-IDF word-importance scores (vocabulary size: 1,445 unique words). The same Logistic Regression classifier is trained on these vectors, in two variants (default and `class_weight='balanced'`).

Variant	Accuracy	Macro F1	Weighted F1
Default	0.828	0.769	0.819
Balanced	**0.861**	**0.866**	0.859

- Class weighting helps **rare categories** (+0.274 F1 on average) far more than **common categories** (+0.016 F1 on average).
- Best-performing category: "Product Issues" (F1 = 1.000, though only 6 test examples — likely an easy, distinctive vocabulary).
- Worst-performing category: "Checkout" (F1 = 0.645) — consistent with the vocabulary-overlap finding from the EDA (Checkout shares ~30% of its vocabulary with Payments).
- **Cost:** essentially free — vectorizing all 1,334 complaints takes ~0.04 seconds; training the classifier takes ~0.06 seconds; no pretrained model, no neural network involved.

---

9. Transformer Approach — Sentence Embeddings + Logistic Regression

(Full notebook: `Transformer/transformer_embeddings_classification.ipynb`)

Method: Each complaint is converted into a 384-dimensional embedding using a pretrained sentence-transformer (`all-MiniLM-L6-v2`, ~22.7 million parameters), which is never trained or fine-tuned on this dataset — only used as a fixed feature extractor. The same Logistic Regression classifier is then trained on these embeddings, again in default and balanced variants.

Variant	Accuracy	Macro F1	Weighted F1
Default	0.851	0.816	0.848
Balanced	**0.881**	**0.889**	0.881

- Class weighting helps **rare categories** (+0.192 F1 on average) far more than **common categories** (+0.019 F1 on average) — the same pattern as the baseline, confirming this is a property of the classifier/data, not the feature method.
- Best-performing category: "Product Issues" (F1 = 1.000).
- Worst-performing category: "Shipping" (F1 = 0.708).
- **Cost:** loading the pretrained model takes ~4 seconds (one-time); generating embeddings for all 1,334 complaints takes ~1.7 seconds; classifier training itself is nearly free (~0.01 seconds, since only the small Logistic Regression layer is trained).

---

10. Combined Comparison — Answering the Research Question

Approach	Accuracy	Macro F1	Feature-extraction cost	Model size
TF-IDF + LogReg (balanced)	0.861	0.866	~0.04s for 1,334 texts	1,445-word vocabulary, no neural network
Transformer + LogReg (balanced)	0.881	**0.889**	~1.7s + ~4s model load	22.7M-parameter neural network

The transformer approach wins on macro F1 (0.889 vs. 0.866, a +0.023 improvement), but the margin is modest relative to the difference in computational cost. The exact ratio of feature-extraction time varies run-to-run (both operations are fast enough that timing noise dominates), but the *category* of cost difference is not in doubt: TF-IDF vectorization involves no neural network at all — just word counting — while the transformer approach must load and run a 22.7-million-parameter pretrained model for every complaint, an order of magnitude more compute regardless of the exact multiplier observed on a given run.

Revisiting the hypothesis: the hypothesis predicted the transformer would help most on rare/overlapping categories. The evidence is mixed on this specific point — the *balancing technique* helped rare categories substantially in **both** approaches (baseline: +0.274, transformer: +0.192), and it is not clearly the case that the transformer's embeddings specifically closed the overlap-

driven confusion (e.g. baseline's weakest category, "Checkout," is exactly the one identified in the EDA as sharing significant vocabulary with "Payments" — a case where semantic understanding *should* help, and the transformer's equivalent categories did see improvement, though a category-by-category deep-dive comparing the two confusion matrices would be needed to confirm this precisely).

Conclusion: for this dataset and task, the transformer approach provides a real but modest accuracy improvement at a substantially higher computational cost. Whether that trade-off is "worth it" depends on the deployment context — for a high-volume, latency-sensitive production system, the classical TF-IDF baseline's near-zero cost may be preferable despite the slightly lower accuracy; for a lower-volume system where every correct routing decision matters more than raw throughput, the transformer's improvement may justify its cost.

---

11. Repository Structure

`Final_Project_Combined.ipynb`, in the root folder, is the primary submission notebook — it merges all three parts below into one continuous, runnable `.ipynb` file (EDA → Baseline → Transformer → Combined Comparison & Conclusion), matching the assignment's request for "the project itself… shared as an .ipynb file." The individual per-person notebooks are kept alongside it for reference and clearer per-teammate attribution.

```
Final Project/
├── Final_Project_Combined.ipynb    (PRIMARY SUBMISSION — all parts combined)
├── PROJECT_DOCUMENTATION.md        (this file)
├── ecommerce_customer_support_cleaned.csv
├── shared_split/                   (the canonical train/test split)
├── EDA/
│   ├── eda_customer_complaints.ipynb
│   └── ecommerce_customer_support_cleaned.csv
├── Baseline/
│   ├── tfidf_baseline_classification.ipynb
│   ├── ecommerce_customer_support_cleaned.csv
│   ├── shared_split/               (train/test indices, loaded from
Transformer/)
│   └── artifacts/
│       ├── logreg_tfidf_balanced.pickle
│       ├── tfidf_vectorizer.pickle
│       └── baseline_metrics.json
└── Transformer/
    ├── transformer_embeddings_classification.ipynb
    ├── ecommerce_customer_support_cleaned.csv
    ├── shared_split/               (the canonical train/test split)
    └── artifacts/
        ├── logreg_embeddings_balanced.pickle
        └── transformer_metrics.json
```

---

12. Tools & Libraries Used

From course: `pandas`, `numpy`, `matplotlib`, `seaborn`, `scikit-learn` (`train_test_split`, `TfidfVectorizer`, `LogisticRegression`, `classification_report`, `confusion_matrix`, `f1_score`), `scipy.stats` (`chi2_contingency`).

Beyond course (research component): `sentence-transformers` (pretrained transformer embeddings via `all-MiniLM-L6-v2`), `psutil` (memory profiling for the cost comparison).